A11yLens — Current Project Handoff

Prepared: 10 November 2026 **Repository:** lakshay74/a11ylens-accessibility-auditor **Current branch:** `main` **Latest repository checkpoint reviewed:** `afe19c5` (6 November 2026) **Project stage:** Week 5 evaluation and interface polish; Week 6 release preparation started

This is an updated status snapshot for continuing the project with Claude. The repository has no newer project commits after 6 November. The last commits added handoff and resume documentation; they did not add new product features. This report separates features that exist in code from checks actually performed and work still pending.

The shareable PDF bundles this update with the earlier technical-report PDF (including its 6 November synchronization supplement) and the resume brief. Those appendices preserve the detailed implementation notes and source-file inventory.

1. Current status at a glance

- `main` was synchronized with `origin/main` at `afe19c5` on 6 November. The repository history has no later commits as of this report date.
- The scanner, local guidance API, retrieval paths, JSON export, evaluation fixtures, maintenance documentation, and local privacy/setup notes are implemented.
- Weeks 1–4 of the roadmap are implemented locally. Week 5 is in progress because repeatable browser evaluation, retrieval/repair outcome measurements, and manual assistive-technology checks are still pending. Week 6 release preparation has started.
- No feature or code changes were recorded after the 6 November handoff checkpoint. This report documents current project state; it does not claim new functionality.
- The working tree also contains untracked local items: `llm-eval-suite/`, `output/pdf/FResume_Updated.pdf`, and `tmp/`. They were not committed or included in this report.

2. What A11yLens is

A11yLens is a Chrome/Brave Manifest V3 browser extension for on-demand accessibility checks. After the user starts a scan, the extension injects a locally bundled axe-core scanner into the active page. It displays axe violations separately from results requiring manual review, helps inspect or highlight affected elements, and can retrieve WCAG guidance for one selected finding through a local FastAPI backend.

The backend returns local guidance without an API key. An optional Groq path can produce a longer explanation and suggested HTML when a developer configures `GROQ_API_KEY` in the backend environment. Groq mode has not been tested with a configured key. Scanning does not require the backend to be online.

Project name: A11yLens — “a11y” is a common numeronym for accessibility; “lens” describes inspecting a page. **Extension version:** 0.1.0. **Distribution status:** local unpacked development build; not published to the Chrome Web Store. **Backend status:** local-only development service; no hosted API or authentication system. **Source license:** none selected for A11yLens. axe-core's separate license notices are retained.

3. Implemented product details

Browser extension

- Manifest V3 popup, background service worker, and content script written in JavaScript, HTML, and CSS.
- Uses `activeTab`, `scripting`, and `storage`; the only host permission is `http://127.0.0.1:8000/*` for the local API.
- Runs locally bundled axe-core 4.14.0 only after a user starts a scan. Configured tags are `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`, and `wcag22aa`; scan requests include `violations` and `incomplete` results.
- Presents violations and manual-review results as separate groups. Manual-review cards use neutral “Possible issue” wording and identify their guidance as review help, not a confirmed failure.

- Summary units distinguish violation rules, affected violation elements, and manual-review rules. When a rule is returned in both categories, the explanatory note is shown once for that rule.
- Finding actions include inspecting a sample target, highlighting it on the page, opening an axe help page only when the URL is a trusted Deque University HTTPS address, and requesting WCAG guidance for the selected finding.
- Limits scan data to 25 nodes per rule; sample HTML is capped at 2,000 characters and axe failure summaries at 1,200 characters. The total affected-node count is retained separately.
- Saves scan results in `chrome.storage.session` so the popup can restore results during the browser session.
- Offers a JSON report export with page title, cleaned URL, counts, and rule summaries. The export excludes HTML, selectors, and failure summaries and removes URL credentials, query parameters, and fragments.
- Detects when the popup is opened as a local file and explains how to load the unpacked extension; the scan button is unavailable outside an extension context.

Backend and retrieval

- Python FastAPI exposes `GET /health` and `POST /explain`.
- Pydantic request validation rejects extra fields and imposes length caps: rule ID 120, description 500, impact 30, HTML sample 2,000, failure summary 1,200, and target selector 500 characters.
- Checked-in WCAG 2.2 A/AA corpus contains 55 concise W3C WAI Understanding summaries: 31 Level A and 24 Level AA. These are informative summaries, not the complete normative WCAG text.
- `backend/rag.py` contains 16 curated axe-rule mappings. Direct mappings are applied first; local TF-IDF lexical retrieval fills remaining slots, with up to three criteria returned.
- Optional local Sentence Transformers (`all-MiniLM-L6-v2`) and FAISS cosine-similarity retrieval is implemented; TF-IDF remains the fallback if optional packages or the model are unavailable.
- Without `GROQ_API_KEY`, explanations use local guidance. With a key, the backend sends the selected finding and retrieved references to Groq for an explanation and suggested HTML. The configured HTTP timeout is 35 seconds. This provider path remains untested.
- The documented local server binds to `127.0.0.1`. The ignored `backend/.env` is the location for a developer's private key; the key is not in extension source.

Project engineering and maintenance

- Public GitHub repository with README, six-week roadmap, project log, changelog, contribution guide, issue templates, architecture notes, setup guide, and a draft privacy disclosure.
- Windows icon regeneration script and branded extension icon assets.
- Build script copies the axe-core bundle and its license/third-party notices into the extension package.
- Local evaluation fixtures include a low-noise baseline and a page with nine intentional candidate issues, together with a repeatable scan procedure and blank results template.
- A keyboard and screen-reader review checklist records browser, assistive technology, zoom, commit, and API mode for future checks.

4. Data flow and privacy boundary

1. User opens the extension popup and explicitly starts a scan.
2. The background service worker checks the active tab and injects local axe-core and the content script.
3. axe-core checks the configured tags in the active page; results are returned to the popup and held in browser session storage.
4. Scan findings are not sent to the local API automatically.
5. If the user requests guidance for one finding, the extension sends that selected finding's rule ID, description, impact, limited HTML sample, failure summary, and selector to FastAPI on localhost.

6. If Groq is configured, the backend can forward selected finding context and retrieved WCAG references to Groq. Otherwise guidance stays local.
7. JSON export is generated locally and leaves out HTML, selectors, and failure summaries.

`docs/PRIVACY.md` is a code-matched local-development draft, not a reviewed store privacy policy. Any hosted service, authentication, provider, retention, or permission change would need a fresh review.

5. Checks and evidence recorded so far

Local guidance API smoke check — 30 October 2026

`GET /health` returned `status: ok` and `groq_configured: false`. Six synthetic `POST /explain` requests completed using local TF-IDF plus curated rule mappings:

Axe rule	Criteria returned by A11yLens	Expected axe 4.14.0 tag reference
`color-contrast`	1.4.3, 1.4.1, 1.4.11	1.4.3 (`wcag143`)
`link-name`	2.4.4, 4.1.2, 2.5.3	2.4.4 (`wcag244`), 4.1.2 (`wcag412`)
`aria-required-children`	1.3.1	1.3.1 (`wcag131`)
`target-size`	2.5.8, 1.4.4	2.5.8 (`wcag258`)
`label-content-name-mismatch`	2.5.3, 1.3.5, 1.3.3	2.5.3 (`wcag253`)
`aria-valid-attr-value`	4.1.2	4.1.2 (`wcag412`)

Each response included its expected axe-tagged criterion; additional criteria came from retrieval. This is a controlled smoke check on synthetic inputs, not retrieval accuracy measured on real findings. Axe WCAG tags can support rule-to-criterion comparison, but they are not independent ground truth for whether a page-level finding is correct. `extension/content.js` currently omits `rule.tags` from its scan payload.

User-reported browser checks

- The user tested the original T1 labels and summary wording on `https://penny-juice.com` and reported that they worked.
- The user later confirmed that one **Get WCAG guidance** request worked in the popup after FastAPI was restarted. This proves one popup-to-local-API request in local mode. The rule ID and whether the card was a violation or manual-review result were not recorded.
- The newest T1 follow-up—the once-per-rule note and manual-review guidance action—still needs its own focused browser check in Brave.
- A screenshot showed A11yLens version 0.1.0 listed in Brave's extension manager; this verifies that the unpacked extension was loaded, not that all scanner behavior was tested.

What these checks do not establish

- No repeatable fixture scan results, retrieval accuracy score, coverage percentage, repair outcome, or performance benchmark are recorded.
- The two fixture pages have not yet been scanned in Brave; the nine listed issues are candidate expectations, not observed axe results.
- No keyboard-only or screen-reader evaluation has been completed. The checklist exists, but its results are blank.
- Groq mode with a configured key has not been tested.
- There is no completed Chrome Web Store, hosted-backend security, or A11yLens source-license review.
- Automated axe results are not a WCAG conformance verdict and cannot detect every accessibility issue.

6. Roadmap status and next work

Completed locally

- **Week 1 — Extension foundation:** Manifest V3 extension, popup, service worker, active-tab messaging, session restoration, and local-file instructions.
- **Week 2 — Accessibility scanning:** bundled axe-core, user-started scans, separate violations/manual-review results, target inspection and highlighting.
- **Week 3 — Explanation API:** strict local FastAPI routes, no-key guidance, optional Groq provider path, setup and error guidance.
- **Week 4 — Retrieval:** linked WCAG summary corpus, curated axe mappings, default TF-IDF search, optional local embeddings/FAISS.
- **Week 5 tasks already implemented:** sanitized JSON export, trusted axe help links, T1 label/count/note/guidance changes, six-rule local API smoke check, GitHub maintenance docs, evaluation fixtures and procedures, assistive-technology checklist.
- **Week 6 work already implemented:** branded icons and Windows regeneration script, third-party notice handling, source permission/data-flow review, and a draft local privacy disclosure.

Recommended next steps

1. In Brave, verify the latest T1 follow-ups on a regular website: note appears once per rule, manual-review cards offer clearly labeled review guidance, and the UI remains understandable for a rule in both categories.
2. Scan the two local evaluation fixtures and record actual violation/manual-review rule IDs and affected elements in `evaluation/RESULTS_TEMPLATE.md`.
3. Fix one intentional issue at a time and record before/after results. Treat axe's tags as a comparison reference and keep human review in the loop.
4. Complete the keyboard and screen-reader checklist and record browser/AT/version/zoom details.
5. Decide whether to retain axe rule tags in scan output if the evaluation needs automated rule-to-criterion comparison; tags do not replace human-verified findings.
6. If desired, configure a private Groq key locally and test its provider path without committing the key. Keep local-only guidance as the default for sensitive page content.
7. Before release, decide on a source license, distribution and backend model, clean-profile screenshots, packaged permission/privacy review, and any Chrome Web Store materials.

7. GitHub history and local run instructions

Public remote: `https://github.com/lakshay74/a11ylens-accessibility-auditor`.

Commit	Date	Recorded work
`d4d241f`	30 Oct	T1 manual-review language and result presentation
`236f35d`	30 Oct	T2 rule-to-criterion mappings and smoke-check results
`e9986f4`	30 Oct	Successful popup guidance check recorded
`5eecc51`	3 Nov	Local evaluation fixtures and scan procedure
`17ed97d`	4 Nov	Safe local backend setup guide
`01f2dbe`	6 Nov	Assistive-technology review procedure
`41dc477`	6 Nov	Source permission and privacy-flow review
`05fdcf1`	6 Nov	GitHub sync documented
`afe19c5`	6 Nov	Resume project handoff added

To load the local version in Brave or Chrome:

```
# Build the locally bundled axe scanner
Set-Location extension
npm install
npm run build
Set-Location ..

# Prepare and start the backend from the repository root
py -m venv backend/.venv
backend/.venv/Scripts/python.exe -m pip install -r backend/requirements.txt
backend/.venv/Scripts/python.exe -m uvicorn main:app --app-dir backend --reload --host 127.0.0.1 --port
8000
```

Then open `brave://extensions` or `chrome://extensions`, enable Developer mode, choose **Load unpacked**, select the repository's `extension/` directory, and scan an ordinary website. Check `http://127.0.0.1:8000/health` for API status. For optional Groq setup and key handling, use `docs/LOCAL_BACKEND_SETUP.md`; never commit `backend/.env`.

8. Source files to inspect

- `README.md` — project overview, setup, privacy notes, and roadmap links.
- `docs/ARCHITECTURE.md` — beginner walkthrough of request and data flow.
- `docs/ROADMAP.md` and `docs/PROJECT_LOG.md` — milestone status, project decisions, and next work.
- `docs/PRIVACY.md` and `docs/LOCAL_BACKEND_SETUP.md` — current data flows and backend/key handling.
- `extension/manifest.json`, `extension/background.js`, `extension/content.js`, `extension/popup.js`, `extension/popup.html`, `extension/popup.css` — browser implementation.
- `backend/main.py`, `backend/rag.py`, `backend/data/wcag.json` — API, retrieval, and WCAG reference corpus.
- `evaluation/README.md`, `evaluation/RESULTS_TEMPLATE.md`, and `evaluation/ASSISTIVE_TECH_TEMPLATE.md` — evaluation procedures and blank records.
- `docs/A11YLENS_PROJECT_PROGRESS_REPORT.md` — previous full technical snapshot, retained in the bundled PDF appendix.
- `docs/RESUME_PROJECT_BRIEF.md` — resume bullets, project framing, verified checks, and a resume-focused Claude prompt.

9. Claude continuation prompt

```
Use the attached A11yLens handoff as the current project record. Treat its distinction between
implemented, user-reported, smoke-checked, and pending items as authoritative. You may inspect the public
GitHub repository and verify claims against its actual code before proposing changes.

Help me continue the project one step at a time. Start by summarizing the current state and recommending
the single highest-value next task from the pending list. Explain the task in beginner-friendly language,
identify the exact files involved, and make a small focused change only after I approve that specific
implementation. Keep commits small and preserve unrelated local/untracked files.

Do not claim WCAG compliance, complete detection, measured accuracy, coverage, performance, successful
repairs, a hosted backend, Chrome Web Store publication, or a chosen A11yLens license. The six API checks
were synthetic smoke checks; one popup guidance request was user-confirmed; the latest T1 UI follow-up,
fixture browser scans, manual assistive-technology review, and Groq mode remain unverified or pending.

For any resume or portfolio wording, ask which implementation work I personally completed and tailor
claims without inventing metrics or sole-ownership claims.
```

10. Earlier detailed appendices

1. Full 20-page technical PDF: the original 30 October snapshot plus a 6 November sync supplement; the Markdown source has the 6 November header. It covers implementation, privacy, history, setup, the WCAG corpus, and rule mappings.
2. Six-page resume brief with resume bullets, project framing, and interview guidance.

A11yLens Project Progress and Technical Handoff

Snapshot date: 30 October 2026

Latest GitHub baseline: `e9a6e50` on `main`

Latest T1 commit: `d4d241f` - Clarify manual review guidance and results (local only)

T2 implementation commit: `236f35d` - Verify local guidance mappings (local only)

T2 result: `link-name` maps to WCAG 2.4.4 and 4.1.2; six local API smoke-check results are recorded below.

Project repository: https://github.com/lakshay74/a11ylens-accessibility-auditor

Project stage: Week 5 evaluation and interface polish; Week 6 release preparation started

PROJECT SNAPSHOT

1. Why this report exists

This is a handoff document for continuing A11yLens with another assistant or developer. It records what exists in the repository, how the parts fit together, decisions already made, what has been observed during setup, and what is still unfinished. The supplied six-week roadmap is tracked in `docs/ROADMAP.md`.

The user later supplied an independent review based on this report and four popup screenshots. That reviewer states that they did not inspect the repository or run the code. Screenshot observations are recorded as reported evidence; implementation comments were checked against the source before T1 changes. The review's task list is treated as recommendations, not as commands by itself.

Items in this report are classified as implemented, observed during setup, or still pending. A checked-off feature means source code or project material exists. It does not mean the feature has passed a formal test program, accessibility evaluation, security review, or store review.

2. Project summary

A11yLens is a Manifest V3 browser extension for inspecting accessibility issues on a page after the user explicitly starts a scan. The extension uses a locally bundled copy of axe-core. It displays automated violations separately from results that axe-core says need manual review, lets the user locate affected elements, and can request WCAG guidance about one selected finding from a local FastAPI service.

The FastAPI service works without an AI provider key. It retrieves local WCAG references and returns a short explanation. If the developer configures a Groq key in the local backend, the service can also request an AI-generated explanation and suggested HTML correction. Scanning does not depend on the backend being online.

The name is **A11yLens**: `a11y` is a common numeronym for accessibility, and `lens` describes looking closely at a page.

Item	Current state
Product	A11yLens - Accessibility Auditor
Browser package	Chrome/Brave Manifest V3 unpacked extension
Extension version	0.1.0
Git branch	`main`
Public source repository	`lakshay74/a11ylens-accessibility-auditor`
Latest commit on GitHub at the time of this report	`e9a6e50` - Link findings to axe rule guidance
T1 local commit after that baseline	`d4d241f` contains the T1 result-label follow-ups and handoff docs; it has not been pushed
T2 local commit	`236f35d` adds the `link-name` criterion mapping and records the six-request local smoke check; it has not been pushed
Backend	Python FastAPI, local-development service
AI provider	Optional Groq API; off unless `GROQ_API_KEY` is configured
WCAG reference set	55 WCAG 2.2 A/AA summaries linked to W3C WAI Understanding pages
Source license	Not selected
Store publication	Not done
Hosted API	Not deployed; local-only development setup

3. What is implemented

3.1 Browser extension foundation

- `extension/manifest.json` declares Manifest V3, the popup, action icons, and a background service worker.
- The extension has the permissions `activeTab`, `scripting`, and `storage`. The only host permission in the manifest is `http://127.0.0.1:8000/*` for the local explanation API.
- The popup is built from `popup.html`, `popup.css`, and `popup.js`. The current design uses a 390-pixel-wide, scrollable popup, a compact branded header, scan panel, status pill, result summary, finding cards, and action buttons.
- The popup shows whether the API is available. When the API is offline, the status says the scanner is ready and API is offline; the user can still scan because scanning runs in the browser.
- Summary tiles identify their units: violation rules, affected violation elements, and manual-review rules. The affected-element count covers violation results.
- Manual-review cards use a neutral **Possible issue** badge that includes axe's impact rating; only violation cards use severity colors. A manual-review card offers **Get WCAG guidance** with a clear note that it is help for reviewing a possible issue, not a confirmed failure. If axe returns the same rule in both groups, a note appears once for that rule.
- The page has status regions for scan and API updates, an alert region for errors, a visible keyboard focus style, and semantic headings and sections. These are implementation choices, not the outcome of a completed accessibility evaluation of the popup.
- The UI presents findings as text created with DOM APIs such as `textContent`; page-derived strings are not injected as popup HTML.
- Extension icons are checked in at 16, 32, 48, and 128 pixels. The Windows PowerShell source for regenerating them is `extension/scripts/build-icons.ps1`.

3.2 On-demand page scanning

1. The user opens the extension popup on an ordinary web page and clicks **Scan this page**.
2. `popup.js` sends `A11YLENS_SCAN_ACTIVE_TAB` to the service worker.
3. `background.js` checks the active tab and rejects unsupported browser pages, including Chrome internal pages, extension pages, the Chrome Web Store, and DevTools pages.
4. Only after the scan request, the worker injects the locally packaged axe-core bundle when needed, then injects `content.js`.
5. `content.js` runs axe-core against the page and returns results to the popup through the worker.
6. The service worker saves the scan under a per-tab key in `chrome.storage.session`, so the popup can restore the same page's scan if the popup closes and reopens during the browser session.

The axe run filters for `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`, and `wcag22aa` tags and requests `violations` and `incomplete` result types. Violations appear as **rule violations**. Incomplete results appear as **manual review** and are not described as confirmed failures.

The three scan summary numbers have different units: `violations` counts violation rules, `affectedElements` counts nodes attached to violation results, and `incomplete` counts manual-review rules. `affectedElements` does not include nodes from incomplete results.

For each rule the content script keeps up to 25 affected nodes in the popup data. Each HTML sample is limited to 2,000 characters and each axe failure summary to 1,200 characters. The total affected-node count is retained separately, so the user can see the count even when only a sample of nodes is kept. The popup offers inspect buttons for up to three sample nodes per finding.

3.3 Finding inspection and help

- **Inspect element** sends the axe target selector to the content script. The content script scrolls the matching element into view, applies a visible outline, and stores the prior inline outline styles.
- **Clear outline** restores those inline outline and outline-offset values.
- The element lookup handles selector paths through open shadow roots where axe provides such a target. A target may no longer exist after page changes; the UI reports that condition. Some frame or shadow-DOM targets may not be highlightable.
- **View sample HTML** opens a details panel for the first captured node. This is a local display aid; the snippet is not included in JSON exports.
- Finding cards include an **Axe rule help** link only if the help URL uses HTTPS and its host is `dequeuniversity.com` or `www.dequeuniversity.com`. The link opens in a new tab with `noopener noreferrer`.
- A manual-review card's badge says **Possible issue** and shows axe's impact in neutral styling. A severity color is reserved for violation cards. Manual-review cards offer WCAG guidance with a disclosure that axe did not confirm a violation. If a rule appears in both result groups, one note explains that both results exist.

3.4 JSON scan export

The results panel includes **Export JSON**. The popup builds a versioned report with `format: "a11ylens-report-v1"`, a scan timestamp, page title, cleaned page URL, summary counts, and a list of findings. Each finding row contains category, axe rule ID, impact, description, affected-element count, and axe help URL.

Before export, the URL's username, password, query string, and fragment are removed. The JSON does not contain page HTML, selectors, or axe failure summaries. The browser creates a local download whose name includes the host and scan date, for example `a11ylens-example.com-2026-10-30.json`. A11yLens does not upload the exported file.

3.5 FastAPI guidance service

- `backend/main.py` creates the FastAPI application and defines `/health` and `POST /explain`.
- The documented development command binds Uvicorn to `127.0.0.1:8000`. No hosted A11yLens API exists in this project snapshot.
- The explanation request is a strict Pydantic model. Unexpected fields are rejected. Text fields have maximum lengths: rule ID 120 characters, description 500, impact 30, HTML 2,000, failure summary 1,200, and selector target 500.
- A request is sent only when a user presses **Get WCAG guidance** on a violation or manual-review card. A manual-review card labels the action as help for reviewing a possible issue and repeats that axe did not confirm a violation in the response. The extension sends the selected axe rule ID, description, impact, one short HTML snippet, one node's failure summary, and target selector to the local API. It does not send the result category. Page scanning itself sends nothing to the backend.
- With no `GROQ_API_KEY`, the service returns locally generated guidance based on retrieved WCAG records. The UI labels this as local guidance and does not pretend that an AI-generated code fix was returned.
- With a key, the API sends the selected finding and retrieved WCAG references from the backend to Groq. The prompt says page HTML is untrusted data, tells the model not to invent criterion numbers or make a conformance claim, and requests a JSON object.
- The backend only returns criterion IDs that were actually in the retrieved set and bounds generated string lengths before responding. It has a 35-second provider request timeout and maps timeout, provider rejection, network failure, and invalid JSON to API errors.
- The health route reports whether Groq is configured. The popup can show **AI explanations ready**, **Local explanations ready**, or **Scanner ready - API offline**.

- CORS accepts the extension origin pattern and local development origins (`localhost` and `127.0.0.1`).

3.6 WCAG retrieval and local reference corpus

- `backend/data/wcag.json` contains 55 records for WCAG 2.2 Level A and AA success criteria. Each record contains an ID, title, conformance level, concise plain-language summary, W3C WAI Understanding source URL, and a note that the source is informative guidance.
- The checked-in set currently contains 31 Level A records and 24 Level AA records. Appendix A lists the exact IDs and titles from the corpus at this snapshot.
- The corpus contains summaries from W3C WAI Understanding pages, not the complete normative WCAG standard. The project documentation says to follow the links and review the applicable criterion before making conformance decisions.
- `backend/scripts/build_wcag_corpus.py` can refresh the corpus. It reads the W3C index and pages, extracts the In Brief guidance, filters to A and AA criteria, retries page requests, sorts records, and writes a temporary file before replacing the corpus file.
- `backend/rag.py` has curated mappings for 16 common axe rule IDs, including `link-name` to 2.4.4 and 4.1.2. Known mappings are added first; the retrieval path then fills up to three criteria using semantic or lexical ranking.
- TF-IDF-style lexical retrieval is the default lightweight path and is available without the optional model packages.
- Optional local semantic retrieval uses Sentence Transformers with `sentence-transformers/all-MiniLM-L6-v2` and FAISS cosine-similarity search. The model is downloaded to the local model cache on first use, and the index is built in memory for the API process. If the packages or model fail to load, the code falls back to TF-IDF.
- `backend/requirements-rag.txt` holds the optional Sentence Transformers and FAISS dependencies separately from the basic backend requirements.

3.7 Local guidance smoke check

On 2026-10-30, the local backend's `/health` endpoint returned `status: ok` with `groq_configured: false`. Six synthetic `POST /explain` requests completed without errors in local mode using `TF-IDF lexical retrieval plus axe rule mapping`:

Axe rule	Criteria returned, in API order	Axe 4.14.0 rule-tag criterion(s)
`color-contrast`	1.4.3, 1.4.1, 1.4.11	1.4.3 (`wcag143`)
`link-name`	2.4.4, 4.1.2, 2.5.3	2.4.4 (`wcag244`), 4.1.2 (`wcag412`)
`aria-required-children`	1.3.1	1.3.1 (`wcag131`)
`target-size`	2.5.8, 1.4.4	2.5.8 (`wcag258`)
`label-content-name-mismatch`	2.5.3, 1.3.5, 1.3.3	2.5.3 (`wcag253`)
`aria-valid-attr-value`	4.1.2	4.1.2 (`wcag412`)

The axe-tagged criterion appears in each response. Additional criteria in a response were supplied by lexical retrieval. The `link-name` mapping was updated after the bundled axe-core 4.14.0 rule table confirmed both `wcag244` and `wcag412`. The official axe-core rule descriptions list the rule tags; they can be used as rule-to-criterion references or for an axe-tag-agreement measure. They are not independent human-verified ground truth for whether a particular page finding is correct. A11yLens currently omits `rule.tags` from scan output because `serializeRule()` in `extension/content.js` does not copy the field. These synthetic calls are a smoke check, not an accuracy evaluation; the Groq path was not exercised.

4. Privacy, security, and third-party handling

4.1 What stays local and what can leave the computer

- Starting a scan runs axe-core in the selected page through the extension. There is no A11yLens hosted service receiving the page.
- To restore results in the popup, `chrome.storage.session` temporarily holds scan data for the browser session. It can include page title and URL, findings, target selectors, HTML samples, and axe failure summaries. This is browser session storage, not a server-side A11yLens account or database.
- Export is generated locally. It removes URL credentials, query parameters, and fragments, and excludes HTML, selectors, and failure summaries.
- Requesting WCAG guidance sends the selected finding to the local FastAPI service. If a Groq key is configured, the selected finding and retrieved references are then sent by that service to Groq. Users should not send a finding if its content is not appropriate to share with the configured provider.
- The current build has no user accounts, analytics, or advertising.
- The draft disclosure is in `docs/PRIVACY.md`; it must be reviewed against the final code and any hosting changes before store publication.

4.2 Security and dependency notes

- The Groq secret belongs in `backend/.env`; `.gitignore` excludes `.env` and `backend/.env`. `backend/.env.example` is the configuration template.
- The extension has no API secret. The browser requests the local API only for an explicitly requested explanation.
- API input fields are strict and length-limited. Results from Groq are size-limited and their criterion IDs are checked against retrieved references.
- The backend development command is bound to loopback. It has not been hardened for public hosting; authentication, quotas, logging, deployment privacy, and provider costs need design review before any hosted deployment.
- `axe-core` is pinned as the npm build dependency (`4.14.0`). `extension/scripts/copy-axe.js` copies the package's scanner bundle and notices into `extension/vendor/` when building. The generated `axe.min.js` is ignored by Git; the license and third-party notices are checked in.
- The repository does not yet have an A11yLens project license. Public repository visibility alone does not grant an explicit reuse license. axe-core has its own separate license and notices.

5. Development and repository structure

Path	Purpose
`extension/manifest.json`	Manifest V3 metadata, permissions, popup, service worker, and icon references
`extension/popup.html`	Popup document structure and accessible status regions
`extension/popup.css`	Popup layout, typography, status styles, focus rings, finding cards, and severity labels
`extension/popup.js`	Popup messaging, rendering, local API calls, export, and local-file preview guard
`extension/background.js`	Active tab validation, on-demand script injection, message handling, and session scan storage
`extension/content.js`	axe-core scan, finding serialization, target highlighting, and outline cleanup
`extension/package.json` and `package-lock.json`	Pinned axe-core build dependency and build command
`extension/scripts/copy-axe.js`	Copies axe-core from npm into the generated extension bundle and copies notices
`extension/vendor/axe-core-LICENSE.txt`	axe-core license notice retained in source
`extension/vendor/axe-core-THIRD-PARTY-NOTICES.txt`	axe-core third-party notices retained in source
`extension/icons/`	Checked-in 16, 32, 48, and 128 pixel PNG icons
`extension/scripts/build-icons.ps1`	Windows script that regenerates the branded PNG icon set
`backend/main.py`	FastAPI routes, request validation, local responses, and optional Groq call
`backend/rag.py`	axe mappings, local TF-IDF ranking, and optional FAISS semantic retrieval
`backend/data/wcag.json`	Checked-in WCAG 2.2 A/AA W3C summary records
`backend/scripts/build_wcag_corpus.py`	W3C corpus refresh script
`backend/requirements.txt`	Basic API dependencies
`backend/requirements-rag.txt`	Optional embedding and FAISS dependencies
`backend/.env.example`	Safe template for local backend configuration
`README.md`	Overview, install/run instructions, privacy summary, status, and license note
`docs/ARCHITECTURE.md`	Beginner-oriented request flow, data shape, retrieval, export, and limits
`docs/ROADMAP.md`	Maintained six-week milestone checklist and release boundary
`docs/PROJECT_LOG.md`	Decisions, current implementation notes, local commands, and next work
`docs/PRIVACY.md`	Draft disclosure aligned to current local data flows
`docs/A11YLENS_PROJECT_PROGRESS_REPORT.md`	Detailed implementation, history, privacy, and roadmap handoff source
`output/pdf/A11yLens_Project_Progress_Report.pdf`	Shareable PDF version of this report
`CONTRIBUTING.md`	Build instructions, coding and privacy guidance, and issue process
`CHANGELOG.md`	User-visible changes recorded in an Unreleased section
`.github/ISSUE_TEMPLATE/bug_report.md`	Bug report template
`.github/ISSUE_TEMPLATE/feature_request.md`	Feature request template
`.gitignore`	Excludes virtual environments, secrets, node modules, and generated axe bundle

6. GitHub and project history

The public repository is `https://github.com/lakshay74/a11ylens-accessibility-auditor`. It was created at the user's request and confirmation. The local `main` branch tracks `origin/main`. Selected implementation commits through the current T2 snapshot:

Commit	Date	Change
`748b32c`	2026-10-27	Initial A11yLens project foundation
`0cb25ca`	2026-10-27	Add A11yLens report export and release docs
`ab56162`	2026-10-27	Clarify local scanning and guidance status
`94d29cc`	2026-10-27	Clarify public repository and local backend status
`108a88c`	2026-10-28	Simplify README project notes
`e9a6e50`	2026-10-28	Link findings to axe rule guidance
`d4d241f`	2026-10-30	Clarify manual review guidance and results
`236f35d`	2026-10-30	Verify local guidance mappings

The Git status was clean at the `e9a6e50` baseline before the handoff report and T1 wording updates. T1 is recorded in local commit `d4d241f`, and T2 is recorded in local commit `236f35d`; neither commit has been pushed. `backend/.env` and API keys are excluded from commits. The public source repository does not mean that a license has been chosen, that the backend is public, or that the extension is published in a browser store.

Work completed during the project so far

- Established the product name, extension foundation, and a structured six-week roadmap.
- Implemented the on-demand scanner, result UI, inspection/highlighting controls, local explanation service, and WCAG reference retrieval.
- Added a versioned JSON export and documentation for architecture, privacy, contributions, changes, roadmap, and project decisions.
- Added branded extension icons, GitHub issue templates, and third-party axe-core license notices.
- Tracked the source in a public GitHub repository at the user's request.
- Refined the setup flow after the popup was first opened as a local file and displayed `Cannot read properties of undefined (reading 'sendMessage')`. The popup now detects when it is not running as an extension, disables the scan action, and explains how to load the unpacked extension in Brave.
- The user showed the extension listed in Brave's Extensions manager as A11yLens - Accessibility Auditor, version 0.1.0. That screenshot confirms extension loading, not scan behavior. The later review separately reports a scan on `claude.ai`; that scan was not reproduced in this session.
- Clarified the README so it describes the public source repository separately from the locally run API, and so it does not imply an unchosen source license.
- Tested the local API health route and six synthetic guidance requests. All six returned the criterion associated with the bundled axe 4.14.0 rule tag; `link-name` returned both 2.4.4 and 4.1.2 after its mapping was corrected.

7. Roadmap status: completed, partial, and pending

Week 1 - Extension foundation: implemented

Manifest V3, popup, background service worker, active-tab messaging, per-session scan storage, and a local-file preview/help state are present.

Week 2 - Accessibility scanning: implemented

axe-core is run on demand for WCAG 2.0, 2.1, and 2.2 A/AA tags. The UI separates violations and incomplete/manual-review results, shows counts and affected element samples, and supports highlighting.

Week 3 - Explanation API: implemented locally; follow-up work remains

The FastAPI API, request limits, no-key local WCAG guidance, and optional Groq explanation path are implemented. Still pending: a safe local API key walkthrough and clearer provider error/retry guidance in the user documentation.

Week 4 - WCAG retrieval: implemented; one setup decision remains

The local 55-record W3C summary corpus, common axe mappings, TF-IDF fallback, and optional local Sentence Transformers plus FAISS path are implemented. Still pending: deciding whether the heavier optional model packages should ever become part of the default developer setup.

Week 5 - Evaluation and interface polish: in progress

Completed: JSON report export, safe Deque axe-help links, GitHub maintenance docs, issue templates, API-offline/local-scan explanation, T1 changes for manual-review labels and guidance, count units, and one-time duplicate-result notes, plus a six-rule local guidance smoke check and the missing `link-name` 4.1.2 mapping.

Still pending:

- Build a repeatable evaluation set to measure detection, retrieval agreement, and repair outcomes. The six local requests used synthetic inputs and are only a smoke check; Groq mode remains untested.
- Review the popup follow-up in Brave: the user's earlier penny-juice.com check covered the original T1 wording, not the latest one-time note and manual-review guidance controls.
- Create a small repeatable set of accessible and intentionally flawed demo pages.
- Record what the scanner catches and whether fixes improve those sample pages, while avoiding claims of automated WCAG conformance.
- Manually review popup keyboard use, screen-reader announcements, and layout with assistive technology.
- Refine report naming and presentation using evaluation feedback.

Week 6 - Release preparation: started

Completed: branded icon assets and a Windows regeneration script, plus a local-development privacy disclosure draft.

Still pending:

- Choose a source license deliberately and confirm the third-party notices for the intended distribution.
- Capture final extension screenshots in a clean browser profile on a demo page, with no personal data visible.
- Review permissions, privacy disclosures, key handling, and deployment risks against the release build.
- Decide whether the backend stays local-only or is redesigned as a secured hosted service.
- Prepare Chrome Web Store materials only if distribution is desired.

8. Current limits and things not yet established

- An automated axe scan is not a WCAG conformance verdict and cannot detect every accessibility problem.
- The corpus is a set of concise, informative W3C WAI Understanding summaries, not the full normative WCAG standard.
- Curated mappings cover common axe rules. Other rules rely on local retrieval and may need manual criterion review.
- AI explanations and suggested HTML use one selected node's limited context. A suggested correction needs a developer's review and may require more page context.
- Frame, shadow DOM, dynamically changing pages, and browser-protected pages can limit scanning or highlighting.
- No demo/repair evaluation set or measured coverage results are currently documented.
- The browser popup and extension have not completed a manual keyboard or screen-reader evaluation in this project record.
- The repo contains no automated test suite. The user reports testing the initial T1 labels on `https://penny-juice.com` successfully. The current popup JavaScript passed `node --check`, but the follow-up that shows the cross-category note once and adds manual-review guidance has not been exercised in Brave. The report PDF has been rendered and visually inspected.
- The attached review reports that four screenshots show a `claude.ai` scan with 3 violation rules, 4 affected violation elements, and 4 manual-review rules. This is a single observed page, not a repeatable evaluation, and this session did not reproduce it.
- The review warns that personal browser details, including an inbox count, private chat titles, and bookmarks, were visible in screenshots. Use a clean browser profile and a seeded demo page for future public captures.
- The review says inspect buttons, sample HTML, and axe-help links were visible, but their behavior was not shown. The export button was visible, but the downloaded report contents were not reviewed.
- The review observed the API-offline status. A later local check confirmed `/health` and local-only guidance for six synthetic rule requests; Groq explanations have not been tested.
- `content.js` serializes axe rule IDs, descriptions, impacts, help URLs, nodes, and counts, but it does not include each rule's `tags` in the scan output. The bundled axe-core 4.14.0 rule descriptions expose criterion tags, including both `wcag244` and `wcag412` for `link-name`. Those tags can serve as rule-to-criterion reference labels or support a tag-agreement metric, but not independent human ground truth for the correctness of a page-level finding. A11yLens would need to retain the tags to compare them with guidance results programmatically.
- No source license, hosted backend, Chrome Web Store release, or formal privacy/store review has been completed.

9. How to run the current local development version

These commands are documented in the repository. They are included here to make the handoff self-contained; they have not been executed as part of preparing this report.

Build the extension scanner bundle

From the repository root in PowerShell:

```
Set-Location extension
npm install
npm run build
```

The build copies `axe.min.js` and its license notices from the installed npm dependency into `extension/vendor/`. The generated JavaScript bundle is ignored by Git and needs to be present locally before loading the unpacked extension.

Install and start the backend

From the repository root:

```
py -m venv backend/.venv
backend/.venv/Scripts/python.exe -m pip install -r backend/requirements.txt
Copy-Item backend/.env.example backend/.env
backend/.venv/Scripts/python.exe -m uvicorn main:app --app-dir backend --reload --host 127.0.0.1 --
port 8000
```

Set `GROQ_API_KEY` in the local `backend/.env` only if AI explanations are wanted. The default model setting is `openai/gpt-oss-20b`; `GROQ_MODEL` can select another model supported by the Groq account. Without a key, local guidance remains available. Optional semantic retrieval can be installed using `backend/requirements-rag.txt`.

The local health URL is `http://127.0.0.1:8000/health`; FastAPI's interactive API documentation is `http://127.0.0.1:8000/docs`.

Load the extension in Brave or Chrome

1. Open `brave://extensions` in Brave or `chrome://extensions` in Chrome.
2. Enable Developer mode.
3. Choose **Load unpacked** and select this repository's `extension` directory.
4. Open an ordinary website, open the A11yLens toolbar popup, and click **Scan this page**.
5. Use **Inspect element**, **Axe rule help**, or **Get WCAG guidance** on a finding. The backend is needed only for the last action.

Opening `extension/popup.html` directly as a file is a static preview, not a running extension. Browser-internal pages, the Web Store, and some protected pages cannot be scanned. If Chrome reports that a content script is unavailable after reloading the extension, reload the website tab as well.

10. Suggested handoff instructions for Claude

Attach this report and share the public repository link. A useful first message is:

This report is the current A11yLens implementation handoff. Please inspect the repository at https://github.com/lakshay74/a11ylens-accessibility-auditor and treat the six-week roadmap in `docs/ROADMAP.md` *as the project's plan. Preserve the distinction between implemented, observed, and pending work. Do not claim accessibility conformance, a completed evaluation, a chosen license, a hosted API, or a store release unless the repository and evidence show that it has happened. Continue with the next unfinished roadmap work and keep the README, roadmap, project log, changelog, and this handoff aligned when project status changes. Explain technical work in beginner-friendly steps.*

11. Repository documents to read next

For more context, read `README.md` for the overview and quick start; `docs/ROADMAP.md` for milestone status; `docs/ARCHITECTURE.md` for request flow; `docs/PROJECT_LOG.md` for decisions and next work; `docs/PRIVACY.md` for the data-flow draft; and `CONTRIBUTING.md` plus `CHANGELOG.md` for maintenance conventions and recorded changes.

Appendix A. WCAG reference corpus inventory

The following 55 IDs and titles are present in `backend/data/wcag.json` at the implementation snapshot. Every record includes its level, short W3C WAI Understanding summary, and source URL. The summaries are informative and do not replace the full WCAG success criteria.

Criterion	Level	Title
1.1.1	A	Non-text Content
1.2.1	A	Audio-only and Video-only (Prerecorded)
1.2.2	A	Captions (Prerecorded)
1.2.3	A	Audio Description or Media Alternative (Prerecorded)
1.2.4	AA	Captions (Live)
1.2.5	AA	Audio Description (Prerecorded)
1.3.1	A	Info and Relationships
1.3.2	A	Meaningful Sequence
1.3.3	A	Sensory Characteristics
1.3.4	AA	Orientation
1.3.5	AA	Identify Input Purpose
1.4.1	A	Use of Color
1.4.2	A	Audio Control
1.4.3	AA	Contrast (Minimum)
1.4.4	AA	Resize Text
1.4.5	AA	Images of Text
1.4.10	AA	Reflow
1.4.11	AA	Non-text Contrast
1.4.12	AA	Text Spacing
1.4.13	AA	Content on Hover or Focus
2.1.1	A	Keyboard
2.1.2	A	No Keyboard Trap
2.1.4	A	Character Key Shortcuts
2.2.1	A	Timing Adjustable
2.2.2	A	Pause, Stop, Hide
2.3.1	A	Three Flashes or Below Threshold
2.4.1	A	Bypass Blocks
2.4.2	A	Page Titled
2.4.3	A	Focus Order
2.4.4	A	Link Purpose (In Context)
2.4.5	AA	Multiple Ways
2.4.6	AA	Headings and Labels
2.4.7	AA	Focus Visible
2.4.11	AA	Focus Not Obscured (Minimum)
2.5.1	A	Pointer Gestures
2.5.2	A	Pointer Cancellation

Criterion	Level	Title
2.5.3	A	Label in Name
2.5.4	A	Motion Actuation
2.5.7	AA	Dragging Movements
2.5.8	AA	Target Size (Minimum)
3.1.1	A	Language of Page
3.1.2	AA	Language of Parts
3.2.1	A	On Focus
3.2.2	A	On Input
3.2.3	AA	Consistent Navigation
3.2.4	AA	Consistent Identification
3.2.6	A	Consistent Help
3.3.1	A	Error Identification
3.3.2	A	Labels or Instructions
3.3.3	AA	Error Suggestion
3.3.4	AA	Error Prevention (Legal, Financial, Data)
3.3.7	A	Redundant Entry
3.3.8	AA	Accessible Authentication (Minimum)
4.1.2	A	Name, Role, Value
4.1.3	AA	Status Messages

Appendix B. Curated axe rule mappings

These are the direct mappings defined by `RULE_TO_CRITERIA` in `backend/rag.py`. Other axe findings are supplemented by TF-IDF or optional FAISS retrieval. A mapping is a starting reference, not an automatic conformance conclusion.

axe rule ID	Mapped WCAG criteria
`image-alt`	1.1.1
`image-redundant-alt`	1.1.1
`role-img-alt`	1.1.1
`color-contrast`	1.4.3
`document-title`	2.4.2
`link-name`	2.4.4, 4.1.2
`heading-order`	1.3.1
`aria-required-children`	1.3.1
`html-has-lang`	3.1.1
`label`	3.3.2, 4.1.2
`select-name`	3.3.2, 4.1.2
`input-button-name`	4.1.2

axe rule ID	Mapped WCAG criteria
`button-name`	4.1.2
`aria-input-field-name`	4.1.2
`aria-hidden-focus`	4.1.2
`duplicate-id-aria`	4.1.2

Handoff reminder: A11yLens is a working local development project, not a released or independently evaluated accessibility product. Update this report when implementation or release status changes.

Follow-up verification

30 October 2026 | Guidance request from the extension popup

After the earlier API smoke checks, FastAPI had been stopped. When the user tried Get WCAG guidance, the popup showed its local-backend connection message. FastAPI was started again on `127.0.0.1:8000`; `/health` returned `status: ok` with Groq not configured. The user then confirmed that the guidance action worked.

WHAT THIS CONFIRMS

One request made through the popup reached the local API and returned guidance. Because `groq_configured` was false, this was the local guidance path.

What was not recorded

The selected axe rule and whether its card was a violation or manual-review result were not recorded. This confirms the popup-to-API connection but does not, by itself, verify the manual-review card wording. Groq-backed responses were not tested.

Restart the local API

From the project root in PowerShell:

```
backend/.venv/Scripts/python.exe -m uvicorn main:app --app-dir backend --host 127.0.0.1 --port 8000
```

Keep that terminal or process running while using the extension. If the popup still shows an offline message after the API starts, close and reopen the popup, then retry.

Local evaluation setup

3 November 2026 | Fixtures created; browser results pending

A small local evaluation set is now in the repository. It includes a low-noise baseline page, a second page with intentional issue examples, a scan procedure, and a blank results template.

The two fixture pages

Fixture	Purpose
accessible-baseline.html	Semantic structure, visible form labels, named controls, readable contrast, and comfortable target sizes. This is not a WCAG conformance claim.
intentional-issues.html	Nine labeled candidate checks: contrast, link name, button name, form label, image alternative, invalid ARIA value, required ARIA child role, target size, and label in name.

Open the pages in Brave

From the repository root in PowerShell, run:

```
backend/.venv/Scripts/python.exe -m http.server 8001 --directory evaluation --bind 127.0.0.1
```

Then open these local URLs and scan each page with the unpacked extension:

```
http://127.0.0.1:8001/demo-pages/accessible-baseline.html
http://127.0.0.1:8001/demo-pages/intentional-issues.html
```

Measurement status

No browser scans or repair outcomes have been recorded yet. The issue-page checklist contains candidate rule IDs based on its markup; it is not an axe result or a measure of scanner accuracy. Record actual violation and manual-review results separately in evaluation/RESULTS_TEMPLATE.md. Review the authored issue and any unexpected result before reporting it. Axe tags can guide rule-to-criterion mapping, but they are not page-level ground truth.

The API is not needed to scan these pages. Start FastAPI separately only when testing Get WCAG guidance. Keep the pages, screenshots, and results local and use a clean browser profile for any public demo.

Local backend and optional Groq

3 November 2026 | Safe setup and error guide

Local WCAG guidance works with no Groq key. The key, if used, belongs in the ignored `backend/.env` file and is read by FastAPI at startup. Never place it in extension files, screenshots, or a commit.

Prepare and start the backend

From the repository root in PowerShell, prepare the environment once. Copy the example file only if `backend/.env` does not already exist; edit an existing file instead of overwriting it.

```
py -m venv backend/.venv
backend/.venv/Scripts/python.exe -m pip install -r backend/requirements.txt
if (-not (Test-Path backend/.env)) { Copy-Item backend/.env.example backend/.env }
```

For local-only mode, leave `GROQ_API_KEY` blank. For Groq mode, put your own key in that file and use a model available to your account. Then stop and restart FastAPI:

```
backend/.venv/Scripts/python.exe -m uvicorn main:app --app-dir backend --reload --host 127.0.0.1 --port 8000
```

Check `http://127.0.0.1:8000/health`. `groq_configured: false` means local guidance is active; `true` means the process found a non-empty key. The key is not returned.

Provider and connection messages

Message	Meaning and action
Popup cannot reach backend	Start FastAPI, check /health, then reopen the popup and retry.
502 - Groq rejected request	Check the key and model in backend/.env, restart, and retry once.
502 - Could not reach Groq	Check network/provider availability and retry when reachable.
502 - invalid explanation	The response could not be parsed. Retry once; if repeated, use local mode and report the problem without key or page data.
504 - timeout	No provider response within 35 seconds. Check availability and retry once. The code does not auto-retry.

Privacy scope

With Groq enabled, the selected finding and retrieved WCAG references go from the local backend to Groq. This can include a short HTML sample and axe failure summary. Use local-only mode when page content should stay on the computer. The Groq path is documented from code but has not been tested with a configured key.

Manual keyboard and screen-reader review

4 November 2026 | Procedure documented; hands-on results pending

A static source review found semantic headings and sections, status and alert regions, standard buttons, links and disclosure controls, and a visible keyboard-focus style. Those are implementation details, not evidence that assistive-technology behavior has passed review.

Repeatable review procedure

The new **evaluation/ASSISTIVE_TECH_TEMPLATE.md** record guides the reviewer through the two local fixture pages with the unpacked extension in Brave. It captures the browser and screen-reader versions, zoom, commit, and API mode so a later run can be compared.

Review area	What to observe	Status
Keyboard use	Tab order, visible focus, scan activation, scrolling, and operation of finding actions and links.	Pending
Announcements	Scan progress, completion, errors, and guidance added after activation.	Pending
Reading order	Popup name, headings, summary, findings, and distinction between violations and manual-review items.	Pending
Fixtures	Run both the baseline and intentional-issues pages; record browser, screen reader, zoom, commit, and API state.	Pending

Current status

The checklist exists, but no keyboard-only or screen-reader run has been recorded. In particular, the reviewer still needs to observe whether results and errors are announced at useful times and whether dynamically added guidance is easy to reach. The roadmap keeps the manual review task open.

A result from these local pages would describe these examples only. It would not establish WCAG conformance or compatibility with every browser, page, or assistive technology.

Permissions, privacy, and notices

6 November 2026 | Static review of repository source

This review compared the checked-in extension manifest, scan flow, API request, key configuration, privacy draft, and axe build script. It records what the current source does; it is not a release certification.

Area	Observed in current source	Status
Browser access	`activeTab` and `scripting` support the user-started active-tab scan; no all-sites host permission is listed.	Done
Storage and API	`storage` restores scan results for the browser session. The only host permission is the loopback API at `127.0.0.1:8000`.	Done
Selected finding	A finding's snippet, selector, and axe summary go to the local API only when the user requests guidance. An optional key enables forwarding to Groq.	Done
Axe notices	The build script copies axe-core 4.14.0's MPL license and third-party notice files into `extension/vendor/`; both are tracked.	Done
Release boundary	No A11yLens source license or hosted API is chosen. Packaged store permissions, privacy declarations, and hosted deployment risks still need review.	Open

Release decisions still needed

The repository is public, but A11yLens has no chosen source license. The backend is local-only in the documented development command and has no authentication; CORS is not a substitute for authentication. Do not expose it as a hosted service until the hosting model, access control, transport, logging, retention, and provider data flow are designed and reviewed.

The privacy disclosure now records the source permission set and still identifies itself as a draft for development. Revisit the final package and store declarations after deciding whether and how to distribute the extension.

GitHub progress sync

6 November 2026 | Sync check at 41dc477, before this handoff update

At the sync check, the eight follow-up commits after baseline **e9a6e50** had been pushed, and local **main** and **origin/main** both pointed to **41dc477**. This page records that checkpoint; the handoff update is a later documentation change.

Commit	Change	GitHub
d4d241f	T1 manual-review labels, summary counts, and guidance.	Pushed
236f35d	T2 WCAG rule mapping and local API checks.	Pushed
2604777	T2 handoff documentation update.	Pushed
e9986f4	Record the user's successful popup guidance check.	Pushed
5eecc51	Local evaluation fixtures and scan procedure.	Pushed
17ed97d	Safe local backend and optional Groq guide.	Pushed
01f2dbe	Manual keyboard and screen-reader review procedure.	Pushed
41dc477	Source permission and privacy flow review.	Pushed

What this sync establishes

The repo now carries the code and documentation history for T1 and T2, the user-confirmed popup check, local evaluation setup, safe backend configuration, the manual assistive-technology checklist, and the source-level permission review. Browser evaluation results and assistive-technology observations are still pending.

A public repository is not a source license, a hosted API, or a browser-store release. Those decisions remain open in **docs/ROADMAP.md**.

PROJECT HANDOFF | RESUME WRITING

A11yLens

On-demand accessibility audit browser extension

PROJECT STATUS	VERSION	SOURCE
Local development; evaluation in progress	0.1.0	GitHub main at 05fdcf1

Project summary

A11yLens is a Manifest V3 browser extension that scans a page after the user starts a scan. It runs axe-core locally, separates violations from results that need manual review, highlights affected elements, and connects one selected finding to WCAG guidance through a local FastAPI service. A Groq explanation path is optional and requires a backend API key.

Resume-ready bullets

- Built A11yLens, a Chrome/Brave Manifest V3 extension that runs axe-core 4.14.0 on demand for WCAG 2.0/2.1 A/AA and 2.2 AA tags, separates violations from manual-review findings, and lets users highlight affected elements.
- Developed a local FastAPI guidance API over 55 WCAG 2.2 A/AA summaries with 16 curated axe-rule mappings and TF-IDF retrieval; implemented optional local Sentence Transformers/FAISS retrieval and a Groq explanation path.
- Implemented a local-first data flow with browser session restoration and JSON export that excludes HTML/selectors and strips URL credentials, queries, and fragments; smoke-checked six synthetic rule requests and one popup-to-local-API request.

One-line project entry
A11yLens | Accessibility Audit Browser Extension - JavaScript, HTML/CSS, Chrome Extensions MV3, axe-core, Python, FastAPI, WCAG retrieval, TF-IDF, optional Sentence Transformers/FAISS, optional Groq API.

Use these as drafts. Claude should tailor them to the role and the candidate's actual contribution without adding unsupported metrics or ownership claims.

How the project works

The user starts the scan in the popup. The extension then injects its local scanner into the active page and sends the results back to the popup.

Step	Part	Responsibility
1	Popup	User starts a scan or chooses an action for a finding.
2	Background service worker	Checks the active tab, injects local scanner scripts, and stores the result in browser session storage.
3	Content script + axe-core	Runs the selected rule tags, serializes violations and incomplete results, and highlights requested elements.
4	Popup results	Separates violations and manual-review results, shows count units, and offers inspect/help/guidance actions.
5	FastAPI	Receives one selected finding when the user asks for WCAG guidance, then retrieves references locally.

Browser extension details

- Manifest V3 with `popup.html`, `popup.css`, `popup.js`, `background.js`, and `content.js`. The extension uses the `activeTab`, `scripting`, and `storage` permissions, with one host permission for `http://127.0.0.1:8000/*`.
- Bundles axe-core 4.14.0 locally. Configured tags are `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`, and `wcag22aa`; axe returns `violations` and `incomplete` results.
- Manual-review findings use a neutral Possible issue badge and explanatory text. The UI does not label an incomplete result as a confirmed failure.
- Keeps at most 25 nodes per rule in scan data; each sample HTML string is capped at 2,000 characters and each axe failure summary at 1,200 characters. The total node count is retained separately.
- Users can inspect up to three sample targets, open an axe help link only when it is an HTTPS Deque University URL, and export a JSON report.
- JSON export includes summary data and rule descriptions but omits HTML, selectors, and failure summaries. It removes URL credentials, query parameters, and fragments.

Limit to state clearly
The configured axe tags define the automated checks; the extension is not a WCAG conformance test and cannot find every accessibility problem. Frame, shadow DOM, dynamic-page, and protected-page behavior can also limit scanning or highlighting.

Guidance service and data flow

Backend stack

Python FastAPI exposes `GET /health` and `POST /explain`. Pydantic request fields are strict and length-limited. FastAPI uses `httpx` for the optional Groq request and `python-dotenv` to load the backend's local environment file.

Request field	Maximum length
Axe rule ID	120 characters
Description	500 characters
Impact	30 characters
HTML sample	2,000 characters
Failure summary	1,200 characters
Target selector	500 characters

Retrieval and optional generation

- The local corpus contains 55 WCAG 2.2 A/AA summaries: 31 Level A and 24 Level AA. Each entry links to a W3C WAI Understanding page; it is informative guidance, not the full normative WCAG text.
- Sixteen curated axe-rule mappings are applied first. TF-IDF lexical retrieval fills remaining slots, with up to three criteria returned.
- Optional local semantic retrieval uses Sentence Transformers (`all-MiniLM-L6-v2`) and FAISS cosine similarity. TF-IDF remains the fallback if optional packages or the model are unavailable.
- Without `GROQ_API_KEY`, the API uses local guidance. With a key, the selected finding and retrieved references are sent from the local backend to Groq. The configured HTTP timeout is 35 seconds.

Privacy boundaries

- Scanning runs locally after the user starts it. Scan restoration data stays in `chrome.storage.session` during the browser session.
- Only a selected finding is sent to FastAPI after the user asks for guidance. If Groq is configured, that selected context can be sent onward to Groq.
- The Groq key stays in ignored `backend/.env`; it is not placed in extension code. The documented API command binds to `127.0.0.1:8000`.
- There is no hosted A11yLens backend or authentication system. The hosted deployment model and source license remain undecided.

What has been checked

Local guidance smoke check, 30 October 2026

The local API health route reported `status: ok` and `groq_configured: false`. Six synthetic requests completed using TF-IDF plus curated axe-rule mappings. Each response included the axe-tagged reference listed below; extra criteria came from retrieval.

Rule	Returned by A11yLens	Axe tag reference
`color-contrast`	1.4.3, 1.4.1, 1.4.11	1.4.3 (`wcag143`)
`link-name`	2.4.4, 4.1.2, 2.5.3	2.4.4 (`wcag244`), 4.1.2 (`wcag412`)
`aria-required-children`	1.3.1	1.3.1 (`wcag131`)
`target-size`	2.5.8, 1.4.4	2.5.8 (`wcag258`)
`label-content-name-mismatch`	2.5.3, 1.3.5, 1.3.3	2.5.3 (`wcag253`)
`aria-valid-attr-value`	4.1.2	4.1.2 (`wcag412`)

How to describe that result

This smoke check shows that six controlled local guidance requests returned the expected axe-tagged criteria. It does not measure retrieval accuracy on real page findings. Axe tags are a reference for rule-to-criterion agreement, not independent ground truth about whether a page-level finding is correct. A11yLens currently omits axe tags from its scan payload.

Popup check reported by the user

The user confirmed one Get WCAG guidance request worked after FastAPI was restarted. This verifies one popup-to-local-API request, but the rule ID and result-card category were not recorded. Groq was not configured. The user separately reported that the earlier T1 labels worked on penny-juice.com; the later UI follow-ups still need a browser check.

Open evaluation work

- The two local fixture pages have not yet been scanned in Brave; the nine listed conditions are candidate expectations, not scan results.
- No repeatable coverage score, retrieval accuracy score, repair outcome, or performance benchmark has been recorded.
- No hands-on keyboard or screen-reader review has been completed. The checklist exists, but its results are blank.
- Groq mode has not been tested with a configured key. The extension has not been published in the Chrome Web Store.

Resume and interview handoff

Thirty-second project explanation

"A11yLens is a browser extension I am building to make accessibility review easier during development. It runs axe-core locally on a page after the user starts a scan, separates violations from items needing manual review, and lets the developer inspect an affected element. For a selected finding, a local FastAPI service retrieves WCAG guidance. The local path is the one I have smoke-checked; the optional Groq path and hands-on accessibility evaluation are still pending."

Useful interview topics

- Why axe incomplete results need human review instead of a failure label.
- How curated rule mappings and lexical search work together, and why the optional embedding path has a fallback.
- How active-tab access and explicit user actions shape the data flow.
- Why axe WCAG tags can help compare mappings but cannot independently verify a page finding.
- What the next evaluation should record: observed rules, category, affected nodes, human review, and repair outcome.

Skills to list only if interview-ready

JavaScript | Browser Extension APIs / Manifest V3 | axe-core | Python | FastAPI | REST/JSON | WCAG 2.2 | TF-IDF retrieval | FAISS (optional implementation) | Git/GitHub

```
Prompt to give Claude
I want to add A11yLens to my resume. Use this brief as the source of truth and tailor the wording to the resume and job description I provide. Give me a concise project heading, one-sentence summary, two or three action-led bullets, a short version for limited space, and interview talking points. Do not invent usage numbers, accuracy, coverage, time saved, performance, deployment, or sole ownership. Describe the six API requests as a synthetic smoke check, not a formal accuracy evaluation. State that Groq, fixture browser scans, and assistive-technology checks remain untested or pending. Do not claim WCAG compliance, Chrome Web Store publication, a hosted backend, or an A11yLens source license. Ask me which parts I personally implemented before making attribution claims.
```

Source files

For deeper review, share `README.md`, `docs/ARCHITECTURE.md`, `docs/ROADMAP.md`, `docs/PROJECT_LOG.md`, `extension/manifest.json`, `extension/background.js`, `extension/content.js`, `extension/popup.js`, `backend/main.py`, `backend/rag.py`, `backend/data/wcag.json`, and `evaluation/README.md`.

All claims in this brief are bounded by the recorded source and checks. Automated findings are not a WCAG conformance verdict.